

Mifare Classic Card API

Development Documentation

V1.0.0

Update Records

Date	Description
2024-07-04	First Edition Release

Contents

- 1 Introduction
 - 1.1 Open
 - 1.2 Close
 - 1.3 Activate
 - 1.4 Read block
 - 1.5 Write block
 - 1.6 Read value
 - 1.7 Write value
 - 1.8 Increment
 - 1.9 Decrement
 - 1.10 Authenticate
 - 1.11 Halt
 - 1.12 Remove

1 Introduction

The Mifare Classic Card interface of the operation class NFC is defined as follows:

1.1 Open

```
/**
 * @Function Open the NFC device
 *
 * @Exception CommonException
 * 1.DeviceAlreadyOpenException: The device has been turned on
 * 2.DeviceNotFoundException: The device cannot be opened
 *
 * */

public synchronized void open() throws CommonException
```

1.2 Close

```
/**
 * @Function Close the NFC device
 *
 * @Exception CommonException
 *
 * */

public synchronized void close() throws CommonException
```

1.3 Activate

```
/**
 * @Function NFC card selection and card reading basic information
 *
 * @Parameter
 * 1.timeOut: Timeout, ms
 *
 * @Exception CommonException
 * 1.IllegalArgumentException: data is invalid
 * 2.DeviceNotOpenException: The device cannot be opened
 * 3.TimeoutException: Timeout
 *
 * @Return byte[]
 *
 * */
```

```
public synchronized byte[] activate(int timeout) throws CommonException
```

1.4 Read block

```
/**
 * @Function NFC M1 card reads the block data
 *
 * @Parameter
 * 1.noBlock: The block area code (0-63) to read is represented in bytes, as the
second block of the first sector is represented as 1.
 *
 * @Exception CommonException
 * 1.IllegalArgumentException: data is invalid
 * 2.DeviceNotOpenException: The device cannot be opened
 *
 * */

public synchronized byte[] m1_read_block(byte noBlock) throws CommonException
```

1.5 Write block

```
/**
 * @Function NFC M1 card writes data to the card block area
 *
 * @Parameter
 * 1.noBlock: The block area code (0-63) to read is represented in bytes, as the
second block of the first sector is represented as 1.
 * 2.inBuf: The data that needs to be written to the card is represented in bytes,
16 bytes in length
 * 3.inLen: The length of the data that needs to be written to the card
 *
 * @Exception CommonException
 * 1.IllegalArgumentException: data is invalid
 * 2.DeviceNotOpenException: The device cannot be opened
 *
 * */

public synchronized void m1_write_block(byte noBlock, byte[] inBuf, int inLen)
throws CommonException
```

1.6 Read value

```
/**
 * @Function Read the value to the M1 card
 *
```

```

* @Parameter
* 1.noBlock: The block area code (0-63) to read is represented in bytes, as the
second block of the first sector is represented as 1.
*
* @Exception CommonException
* 1.IllegalArgumentException: data is invalid
* 2.DeviceNotOpenException: The device cannot be opened
*
* */

public synchronized byte[] m1_read_value(byte noBlock) throws CommonException

```

1.7 Write value

```

/**
* @Function Read the value to the M1 card
*
* @Parameter
* 1.noBlock: The block area code (0-63) to read is represented in bytes, as the
second block of the first sector is represented as 1.
* 2.inBuf: The data that needs to be written to the card is represented in bytes,
16 bytes in length
* 3.inLen: The length of the data that needs to be written to the card
*
* @Exception CommonException
* 1.IllegalArgumentException: data is invalid
* 2.DeviceNotOpenException: The device cannot be opened
*
* */

public synchronized void m1_write_value(byte noBlock, byte[] inBuf,int inLen)
throws CommonException

```

1.8 Increment

```

/**
* @Function M1 card adds value (addition)
*
* @Parameter
* 1.srcAddr: Source block address, 1 bytes
* 2.destAddr: Target block address, 1 bytes
* 3.inBuf: Values are represented in an array of bytes
* 4.inLen: Length of byte array
*
* @Exception CommonException
* 1.IllegalArgumentException: data is invalid
* 2.DeviceNotOpenException: The device cannot be opened
*
* */

```

```
public synchronized void m1_increment(byte srcAddr, byte destAddr, byte[] inBuf,
int inLen) throws CommonException
```

1.9 Decrement

```
/**
 * @Function M1 card value reduction (M1 card subtraction)
 *
 * @Parameter
 * 1.srcAddr: Source block address, 1 bytes
 * 2.destAddr: Target block address, 1 bytes
 * 3.inBuf: Values are represented in an array of bytes
 * 4.inLen: Length of byte array
 *
 * @Exception CommonException
 * 1.IllegalArgumentException: data is invalid
 * 2.DeviceNotOpenException: The device cannot be opened
 *
 * */

public synchronized void m1_decrement(byte srcAddr, byte destAddr, byte[] inBuf,
int inLen) throws CommonException
```

1.10 Authenticate

```
/**
 * @Function Card sector password check
 *
 * @Parameter
 * 1.noBlock: Block size code (0-63) to verify
 * 2.passwdType: Type of password to check, keyA (0x0A) or keyB (0x0B), two types
 * 3.passwd: The password to be checked is represented as an array of bytes
 *
 * @Exception CommonException
 * 1.IllegalArgumentException: data is invalid
 * 2.DeviceNotOpenException: The device cannot be opened
 *
 * */

public synchronized void m1_authenticate(byte noBlock, byte passwdType, byte[]
passwd) throws CommonException
```

1.11 Halt

```
/**
 * @Function Hang up the card
 *
 * @Exception CommonException
 * 1.IllegalArgumentException: data is invalid
 * 2.DeviceNotOpenException: The device cannot be opened
 *
 * */

public synchronized void halt() throws CommonException
```

1.12 Remove

```
/**
 * @Function Waiting for card removal
 *
 * @Parameter
 * 1.timeOut: Timeout, ms
 *
 * @Exception CommonException
 * 1.IllegalArgumentException: data is invalid
 * 2.DeviceNotOpenException: The device cannot be opened
 *
 * */

public synchronized void remove(int timeOut) throws CommonException
```